

```

1 import torch
2 from torch.utils.data import DataLoader, TensorDataset
3
4
5 X = torch.randn(10240, 8, dtype=torch.float32)
6
7
8 true_w = torch.randn(8,1)
9 true_b = torch.randn(1,)
10 Y = X @ true_w + true_b + torch.randn(10240, 1) * 0.01
11
12
13 # 1. 定义损失函数和优化器
14 w = torch.randn((8, 1), requires_grad=True)
15 b = torch.tensor([0.0], requires_grad=True)
16
17 # 2. 定义损失函数
18 loss_fn = torch.nn.MSELoss()
19 optimizer = torch.optim.SGD([w, b], lr=0.01)
20
21 # 3. 设置超参数
22 batch_size = 32 # 批次大小 10240 个样本分成 320 个批次
23 epochs = 50
24
25 # 4. 数据加载器
26 dataset = TensorDataset(X, Y)
27 loader = DataLoader(dataset, batch_size=batch_size, shuffle=True)
28
29 # 5. 训练循环
30 for epoch in range(epochs):
31     epoch_loss = 0
32     for x_batch, y_batch in loader:
33         # 计算预测值
34         y_pred = x_batch @ w + b
35
36         # 计算损失
37         loss = loss_fn(y_pred, y_batch)
38
39         # 反向传播
40         optimizer.zero_grad()
41         loss.backward()
42
43         # 更新参数
44         optimizer.step()
45
46         epoch_loss += loss.item()
47
48     if (epoch + 1) % 10 == 0:
49         print(f"Epoch [{epoch+1}/{epochs}], Loss: {epoch_loss/len(loader):.4f}")
50
51 # 6. 打印结果
52 print("\n训练完成")
53 print(f"最终权重 w: {w.detach().numpy()}")
54 print(f"最终偏置 b: {b.item():.4f}")

```